

PROM-Consistent Learned Reduced Models for a Parametric 2D Burgers Benchmark

Abstract

This manuscript studies learned reduced models at the PROM level for a two-dimensional parametric Burgers benchmark. A fixed MLSPG-sensitive basis of dimension $n_{\text{tot}} = 151$ defines the coefficient space, and the reference coefficient trajectories are produced by a linear PROM in that full reduced space. We compare three intrusive PROM-ANN closures, a PROM-POD-AE nonlinear-manifold solver, a direct POD-NN-ROM map, and a POD-DL-ROM latent map. Two data regimes are considered. The baseline regime uses the original nine PROM training trajectories. The enriched regime keeps the same basis and the same PROM formulation, but augments the coefficient data with 36 additional PROM trajectories selected by Latin hypercube sampling. The enriched campaign substantially reduces coefficient regression error and brings all learned PROM-ROM state predictions close to the linear PROM accuracy level at the four evaluation points. The purpose of this document is to establish a clean PROM-consistent baseline before adding any residual-sampling approximation.

1 Objective and Scope

The goal is to answer a deliberately restricted question: given a fixed linear PROM space, can learned nonlinear closures reproduce the linear PROM coefficient dynamics and state accuracy? This is separated from online residual sampling. All intrusive results in this document minimize the full PROM residual, and all non-intrusive results are reconstructed in the same 151-dimensional reduced basis.

This separation is useful for two reasons. First, it isolates the learning error from any additional quadrature or sampling error. Second, it makes the linear PROM coefficient trajectory a well-defined teacher for the learned models. If a learned model fails at this level, the failure is caused by either the learned coefficient map or by its feedback through the nonlinear PROM solve, not by residual sampling.

2 PROM Reference Problem

Let $\mathbf{u}(t; \boldsymbol{\mu}) \in \mathbb{R}^{N_h}$ denote the HDM state for parameters $\boldsymbol{\mu} = (\mu_1, \mu_2)$. The reduced approximation uses the fixed affine trial space

$$\mathbf{u}(t; \boldsymbol{\mu}) \approx \mathbf{u}_{\text{ref}} + \mathbf{V}_{151} \mathbf{q}(t; \boldsymbol{\mu}), \quad \mathbf{V}_{151} \in \mathbb{R}^{N_h \times 151}. \quad (1)$$

The basis $\mathbf{V}_{151}$ is the MLSPG-sensitive basis from the current PROM campaign. In contrast with a purely Euclidean snapshot POD basis, this basis was selected to be compatible with the least-squares residual solve. The basis is fixed throughout both the baseline and enriched experiments; enrichment changes only the coefficient data used to train the neural models.

At each time step, the linear PROM solves a least-squares residual problem in the full reduced space,

$$\mathbf{q}_{151}^{k,\text{lin}}(\boldsymbol{\mu}) = \arg \min_{\boldsymbol{\eta} \in \mathbb{R}^{151}} \left\| \mathbf{R}(\mathbf{u}_{\text{ref}} + \mathbf{V}_{151}\boldsymbol{\eta}; \mathbf{u}^{k-1}, \boldsymbol{\mu}, t_k) \right\|_2^2. \quad (2)$$

The resulting trajectory $\mathbf{q}_{151}^{\text{lin}}(t; \boldsymbol{\mu})$ is used as the coefficient reference for training and for coefficient-space diagnostics. State errors are always measured against the HDM snapshots,

$$e_u(\boldsymbol{\mu}) = 100 \frac{\|\mathbf{u}_{\text{HDM}} - \mathbf{u}_{\text{ROM}}\|_F}{\|\mathbf{u}_{\text{HDM}}\|_F}. \quad (3)$$

Coefficient errors are measured against the linear PROM teacher,

$$e_q(\boldsymbol{\mu}) = 100 \frac{\|\mathbf{q}_{151}^{\text{lin}} - \hat{\mathbf{q}}\|_F}{\|\mathbf{q}_{151}^{\text{lin}}\|_F}. \quad (4)$$

3 Learned Model Families

The learned models all use the same fixed coefficient space, but differ in which coordinates are solved by the PROM residual and which coordinates are predicted.

PROM-ANN Case 1. The PROM solves the first ten coordinates $\mathbf{q}_p = (q_1, \dots, q_{10})$. The secondary coordinates $\mathbf{q}_s = (q_{11}, \dots, q_{151})$ are predicted from the solved primary coordinates,

$$\mathbf{q}_s = \mathcal{N}_1(\mathbf{q}_p). \quad (5)$$

This closure is state-only. It is intrusive because $\mathbf{q}_p$ is determined by a residual minimization, but it does not receive $\boldsymbol{\mu}$ or t explicitly.

PROM-ANN Case 2 and POD-NN-ROM. A master map predicts the full coefficient vector from parameters and time,

$$\hat{\mathbf{q}}_{151} = \mathcal{N}_2(\mu_1, \mu_2, t). \quad (6)$$

The same map is used in two ways. With $n = 0$, it is the purely non-intrusive POD-NN-ROM. With $0 < n < 151$, the PROM solves $(q_1, \dots, q_n)$ by least squares and injects the tail $(\hat{q}_{n+1}, \dots, \hat{q}_{151})$ from the master map. With $n = 151$, the method becomes the linear PROM in Eq. (2).

PROM-ANN Case 3. The PROM again solves $\mathbf{q}_p = (q_1, \dots, q_{10})$, but the secondary closure receives both the solved primary coordinates and the parameter-time coordinates,

$$\mathbf{q}_s = \mathcal{N}_3(\mathbf{q}_p, \mu_1, \mu_2, t). \quad (7)$$

This is the most informative ANN closure among Cases 1–3 because it combines residual-solved coordinates with explicit parametric context.

PROM-POD-AE. An autoencoder is trained on the linear PROM coefficient trajectories. Its decoder defines a nonlinear coefficient manifold,

$$\mathbf{q}_{151} \approx \mathcal{D}(\mathbf{z}), \quad \mathbf{z} \in \mathbb{R}^{10}. \quad (8)$$

The online unknown is $\mathbf{z}$, and the PROM residual is minimized over this latent coordinate.

POD-DL-ROM. The POD-DL-ROM is non-intrusive. It learns a map from (μ_1, μ_2, t) to a latent coordinate $\mathbf{z}$, then decodes $\mathbf{z}$ to $\mathbf{q}_{151}$. This tests whether the parameter-time map is easier in a learned latent space than in all 151 coefficients.

4 Data Sets and Evaluation Points

The baseline training set is the original 3×3 parameter grid, giving nine PROM coefficient trajectories. Two additional PROM trajectories are used only as external validation data for the parameter-time networks; these validation points are not part of the four test points reported below. The four evaluation points are

$$\begin{aligned}\boldsymbol{\mu}^{(v)} &= (4.875, 0.0225), & \boldsymbol{\mu}^{(1)} &= (4.560, 0.0190), \\ \boldsymbol{\mu}^{(2)} &= (5.190, 0.0260), & \boldsymbol{\mu}^{(3)} &= (4.000, 0.0330).\end{aligned}$$

Here $\boldsymbol{\mu}^{(v)}$ is the verification point, $\boldsymbol{\mu}^{(1)}$ and $\boldsymbol{\mu}^{(2)}$ are off-grid in-domain points, and $\boldsymbol{\mu}^{(3)}$ is an extrapolatory point.

The enriched data set keeps the same basis $\mathbf{V}_{151}$ and the same PROM solver, but adds coefficient trajectories at 36 new parameter points. Eighteen are interior Latin-hypercube samples in the original training rectangle, and 18 are exterior Latin-hypercube samples in a rectangle expanded by a 25% margin. The LHS seed is fixed to 42. No additional HDM snapshots are used to train these coefficient maps; the additional targets are generated by the linear PROM teacher.

Table 1: PROM coefficient data used for the enriched training campaign.

Quantity	Value
Baseline PROM trajectories	9
Interior LHS trajectories	18
Exterior LHS trajectories	18
Total training trajectories	45
Snapshots per trajectory	501
Training samples	22545
LHS seed	42
Margin fraction	0.25

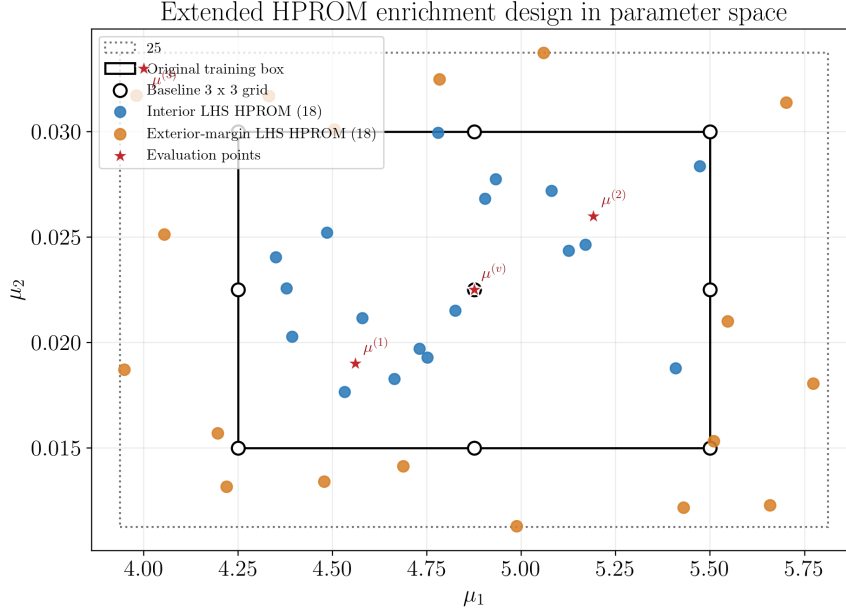


Figure 1: Parameter locations for the enriched PROM coefficient data set. The 36 additional trajectories are used only for training the learned maps; the four reported evaluation points remain unchanged.

5 Baseline PROM-Only Results

Table 2 reports the selected baseline training runs. The largest validation error occurs for the master parameter–time map because it predicts all 151 coefficients from only (μ_1, μ_2, t) . The Case 1, Case 3, and autoencoder tasks are easier because their inputs contain coefficient information.

Table 2: Baseline PROM-only Stage-3 selected networks. Errors are relative Frobenius percentages in coefficient space.

Model	learned map	parameters	train (%)	validation (%)
PROM-ANN Case 1	$(q_1, \dots, q_{10}) \mapsto (q_{11}, \dots, q_{151})$	564,621	1.130	1.251
PROM-ANN Case 3	$(q_1, \dots, q_{10}, \mu_1, \mu_2, t) \mapsto (q_{11}, \dots, q_{151})$	565,389	0.405	0.482
Master POD-NN-ROM (Case 2 tail source)	$(\mu_1, \mu_2, t) \mapsto (q_1, \dots, q_{151})$	565,399	2.601	4.391
PROM-POD-AE	$q_{1:151} \mapsto z_{1:10} \mapsto q_{1:151}$	486,817	0.106	0.127
POD-DL-ROM	$(\mu_1, \mu_2, t) \mapsto z_{1:10} \mapsto q_{1:151}$	952,747	0.810	0.935

Table 3 gives the baseline online state error. The linear PROM is not a theorem-level lower bound against the HDM, but it is the reference accuracy level associated with the fixed 151-dimensional PROM basis and teacher data. The intrusive learned PROMs are close to that reference for in-domain points. The purely non-intrusive maps degrade more strongly, especially at the extrapolatory point.

Table 3: Baseline PROM-only online errors $100 \|\mathbf{u}_{\text{HDM}} - \mathbf{u}_{\text{ROM}}\|_F / \|\mathbf{u}_{\text{HDM}}\|_F$. The mean column averages $\mu^{(v)}, \mu^{(1)}, \mu^{(2)}$.

Model	$\mu^{(v)}$	$\mu^{(1)}$	$\mu^{(2)}$	mean	$\mu^{(3)}$	time (s)
Linear PROM	0.413	0.460	0.472	0.448	0.868	1289.0
PROM-ANN Case 1	0.409	0.520	0.683	0.537	1.584	222.8
PROM-ANN Case 2, $n = 10$	1.013	1.427	1.670	1.370	1.898	91.1
PROM-ANN Case 2, $n = 20$	0.999	1.328	1.505	1.278	1.658	124.1
PROM-ANN Case 3	0.405	0.521	0.576	0.501	1.392	224.2
PROM-POD-AE, $n_z = 10$	0.408	0.539	0.607	0.518	1.978	231.4
POD-NN-ROM	1.036	1.675	1.842	1.518	3.214	0.003
POD-DL-ROM, $n_z = 10$	0.489	1.398	1.463	1.116	3.728	0.006

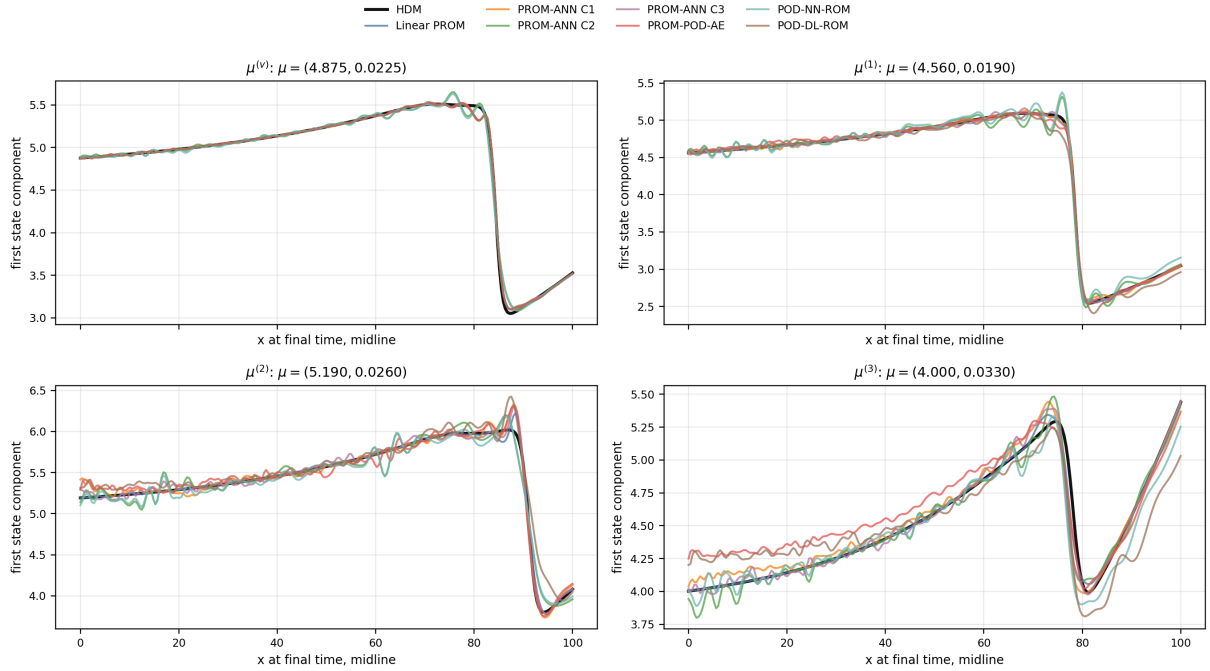


Figure 2: Baseline PROM-only final-time midline comparison against the HDM at the four evaluation points. All curves are computed from saved solver-side snapshots.

6 Case 2 Mechanism: From POD-NN-ROM to Linear PROM

Case 2 gives a useful diagnostic because it connects the direct parameter-time map to the full linear PROM. We run

$$n \in \{0, 3, 5, 10, 20, 30, 50, 100, 151\}, \quad (9)$$

where $n = 0$ is the POD-NN-ROM prediction and $n = 151$ is the linear PROM. For intermediate n , LSPG solves the first n coordinates while the master ANN supplies the remaining coordinates.

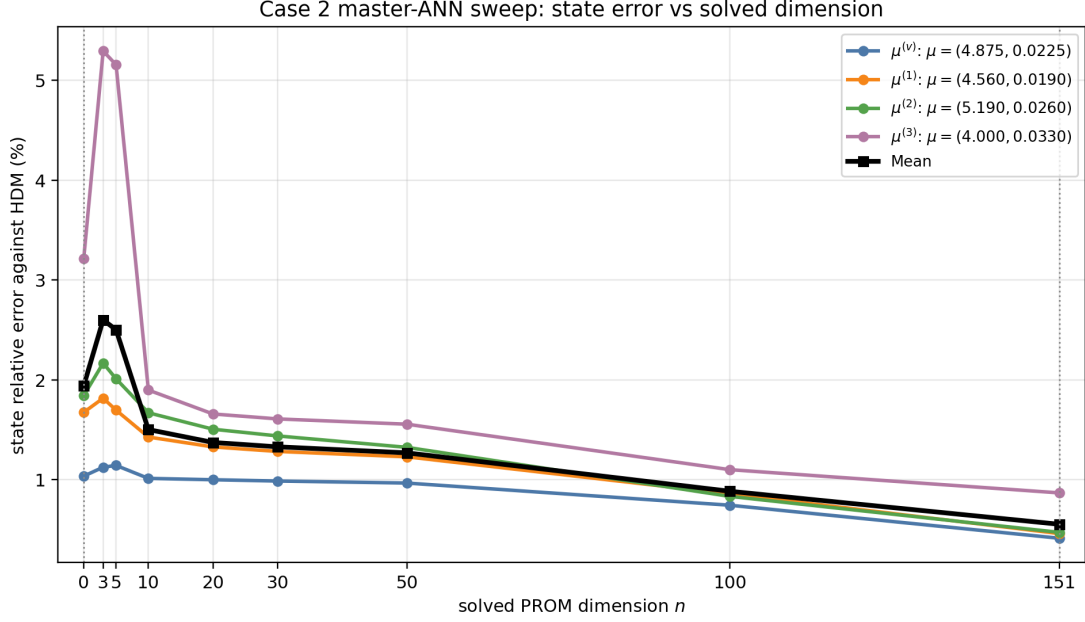


Figure 3: Baseline Case 2 state error as a function of the number of solved primary coordinates. The left endpoint is the direct POD–NN–ROM map and the right endpoint is the linear PROM.

The sweep shows that solving more primary coordinates does not automatically produce monotone improvement at all points. The injected tail still affects the residual minimization, and the benefit of increasing n depends on whether the remaining predicted tail is accurate enough for that parameter point.

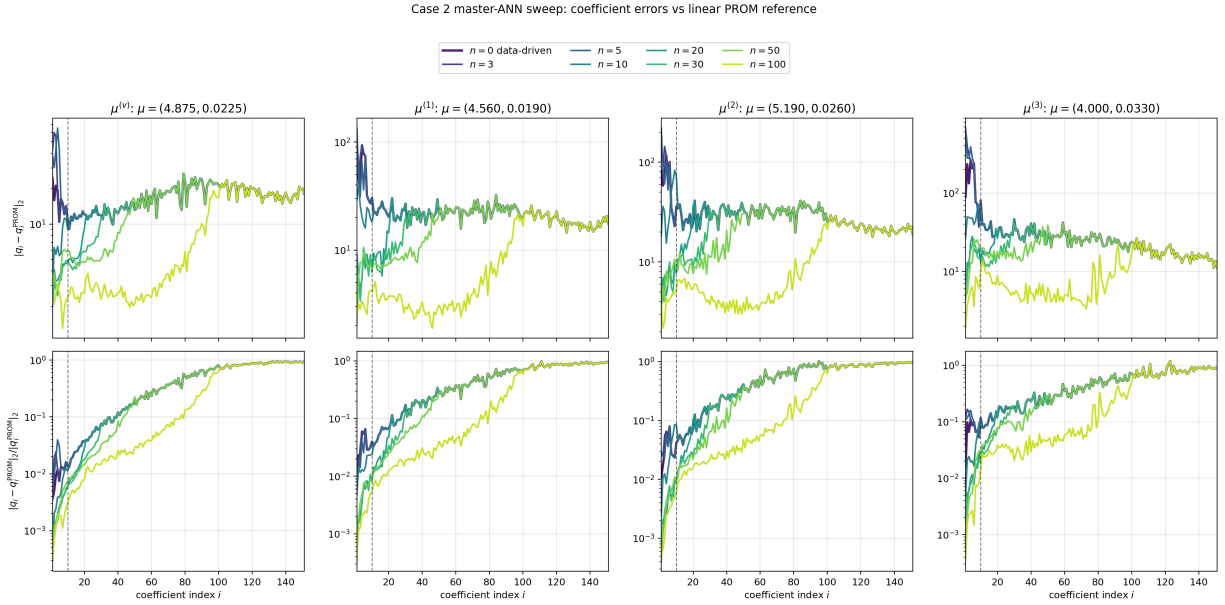


Figure 4: Baseline Case 2 coefficient errors with respect to the linear PROM coefficient trajectories for the same n -sweep. The vertical line marks coefficient 10, the primary dimension used in the main Case 2 runs.

To separate the role of the injected secondary coordinates from the LSPG solve itself, we also perturb the secondary coordinates around the linear PROM values for the fixed $n = 10$ split. This diagnostic asks how much primary-coordinate and state error is induced when the secondary tail is no longer exact.

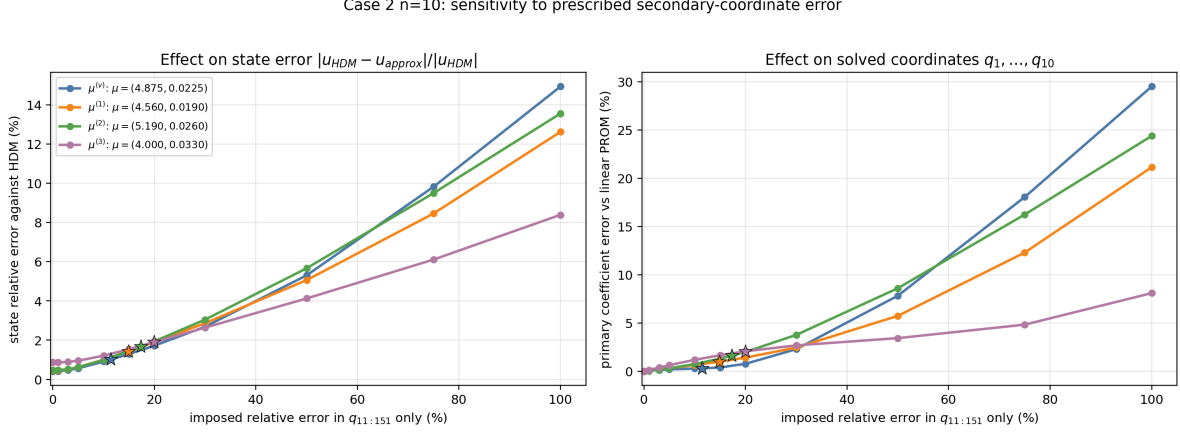


Figure 5: Baseline Case 2 secondary-coordinate perturbation diagnostic for $n = 10$. The imposed secondary-coordinate error is measured relative to the linear PROM tail; the output curves report the induced state and primary-coordinate errors.

The interpretation is that Case 2 is not only a regression problem for the tail. Tail errors can change the solved primary coordinates, so the online error reflects both tail-prediction error and residual-solve feedback.

7 Baseline Online Coefficient Diagnostics

The online state error is the final metric, but coefficient-space diagnostics explain where the learned models lose accuracy. Table 4 reports coefficient errors from the actual online outputs. Case 1, Case 2, Case 3, and PROM-POD-AE use their PROM online trajectories, while POD-NN-ROM and POD-DL-ROM use their direct non-intrusive predictions. All coefficient errors are measured against the linear PROM trajectory with $n_{\text{tot}} = 151$. For Case 1 and Case 3, the online solver did not save $\mathbf{q}(t)$ directly, so the coefficients are recovered from the saved PROM state snapshots by

$$\mathbf{q}(t) = \arg \min_{\eta \in \mathbb{R}^{151}} \|\mathbf{V}_{151}\eta - (\mathbf{u}_{\text{PROM}}(t) - \mathbf{u}_{\text{ref}})\|_2. \quad (10)$$

Table 4: Baseline online coefficient error against the linear PROM with $n_{\text{tot}} = 151$. Values are relative Frobenius percentages over all 151 coefficients and all time steps.

Point	Case 1	Case 2 $n = 10$	Case 2 $n = 20$	Case 3	POD-AE	POD-NN-ROM	POD-DL-ROM
$\mu^{(v)}$	0.295	2.440	2.401	0.164	0.140	2.499	0.814
$\mu^{(1)}$	0.593	3.361	3.123	0.856	0.773	3.924	3.129
$\mu^{(2)}$	1.297	3.906	3.512	0.788	0.929	4.249	3.225
$\mu^{(3)}$	2.923	4.257	3.662	2.396	4.394	7.444	8.639

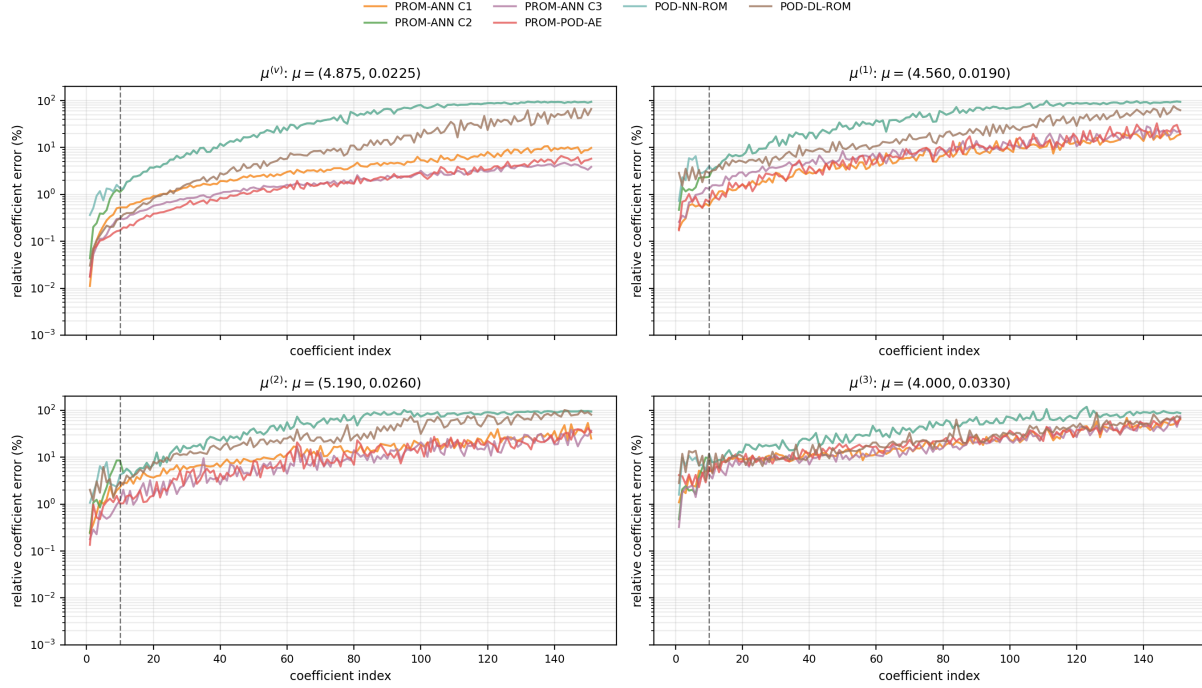


Figure 6: Baseline online per-coefficient relative errors against the $n_{\text{tot}} = 151$ linear PROM coefficient trajectory. The vertical line marks coefficient 10. The linear PROM itself is the reference and is not plotted.

The high-index coefficients often show large relative errors because their trajectory norms are small. This does not mean that each high-index coefficient dominates the state error. It does show, however, that the map $(\mu_1, \mu_2, t) \mapsto \mathbf{q}_{151}$ is data-limited in the baseline regime: the high-index tail is low energy but still difficult to interpolate from only nine parameter trajectories.

8 Effect of PROM Data Enrichment

The enriched campaign tests whether the baseline degradation is primarily an architecture issue or a data-coverage issue. The architecture is kept fixed. The basis $\mathbf{V}_{151}$, validation points, online evaluation points, and online PROM formulations are also unchanged. Only the training coefficient data are expanded from nine PROM trajectories to 45 PROM trajectories.

Table 5: Training and validation coefficient errors before and after PROM data enrichment. The validation set is kept fixed when available, so the reduction measures generalization improvement rather than a change in evaluation criterion.

Model	Base train	Base val.	Enriched train	Enriched val.	Val. reduction
Case 1	1.130	1.251	0.360	0.374	70.1%
Case 2	2.601	4.391	0.435	0.555	87.4%
Case 3	0.405	0.482	0.230	0.268	44.5%
POD-AE	0.106	0.127	0.054	0.074	42.0%
POD-DL-ROM	0.810	0.935	0.142	0.158	83.1%

The strongest gain is obtained by the master Case 2/POD–NN map: its validation error decreases from 4.391% to 0.555%. This is the clearest evidence that the previous difficulty was largely caused by the sparsity of the nine-trajectory parameter training set, not by an intrinsic impossibility of learning the map in this benchmark. The POD–DL map also improves strongly, while the already accurate autoencoder still benefits.

Table 6: Online state errors before and after PROM data enrichment. The linear PROM row is unchanged and is shown as the fixed reference accuracy level.

Model	Data	$\mu^{(v)}$	$\mu^{(1)}$	$\mu^{(2)}$	Mean	$\mu^{(3)}$	Time (s)
Linear PROM	reference	0.413	0.460	0.472	0.448	0.868	1289.0
Case 1	base	0.409	0.520	0.683	0.537	1.584	222.8
Case 1	enriched	0.411	0.459	0.469	0.446	0.868	226.7
Case 2	base	1.013	1.427	1.670	1.370	1.898	91.1
Case 2	enriched	0.435	0.477	0.496	0.469	0.872	81.8
Case 3	base	0.405	0.521	0.576	0.501	1.392	224.2
Case 3	enriched	0.412	0.458	0.470	0.447	0.869	224.1
POD-AE	base	0.408	0.539	0.607	0.518	1.978	231.4
POD-AE	enriched	0.412	0.459	0.471	0.447	0.871	226.8
POD-NN-ROM	base	1.036	1.675	1.842	1.518	3.214	0.003
POD-NN-ROM	enriched	0.437	0.479	0.500	0.472	0.897	0.005
POD-DL-ROM	base	0.489	1.398	1.463	1.116	3.728	0.006
POD-DL-ROM	enriched	0.414	0.460	0.473	0.449	0.868	0.006

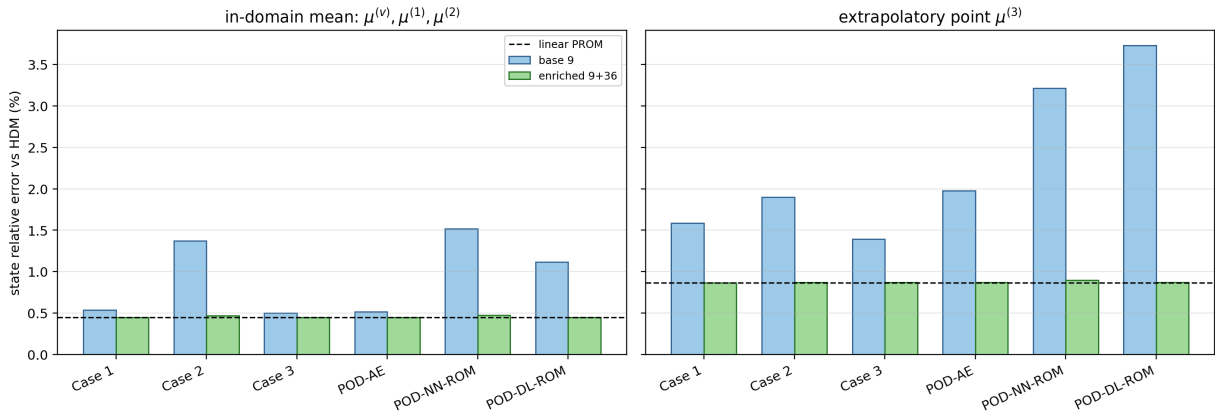


Figure 7: Effect of PROM data enrichment on online state errors. Left: mean error over the verification and two off-grid in-domain points. Right: extrapolatory point.

After enrichment, all learned models are close to the linear PROM state accuracy at the reported points. The most important qualitative change is that the non-intrusive maps no longer behave as outliers: the enriched POD–NN–ROM and POD–DL–ROM have state errors comparable to the intrusive learned PROMs. For the extrapolatory point, the enriched errors also collapse near the

fixed linear PROM level, indicating that the exterior LHS samples provide useful information in the expanded parameter region.

Table 7: Online coefficient errors against the linear PROM teacher before and after PROM data enrichment. Values are relative Frobenius percentages in coefficient space.

Model	Data	$\mu^{(v)}$	$\mu^{(1)}$	$\mu^{(2)}$	Mean	$\mu^{(3)}$
Case 1	base	0.295	0.593	1.297	0.729	2.923
Case 1	enriched	0.066	0.057	0.089	0.070	0.178
Case 2	base	2.440	3.361	3.906	3.236	4.257
Case 2	enriched	0.377	0.357	0.433	0.389	0.583
Case 3	base	0.164	0.856	0.788	0.603	2.396
Case 3	enriched	0.039	0.038	0.055	0.044	0.117
POD-AE	base	0.140	0.773	0.929	0.614	4.394
POD-AE	enriched	0.041	0.041	0.059	0.047	0.151
POD-NN-ROM	base	2.499	3.924	4.249	3.557	7.444
POD-NN-ROM	enriched	0.385	0.362	0.446	0.398	0.805
POD-DL-ROM	base	0.814	3.129	3.225	2.389	8.639
POD-DL-ROM	enriched	0.108	0.099	0.130	0.112	0.240

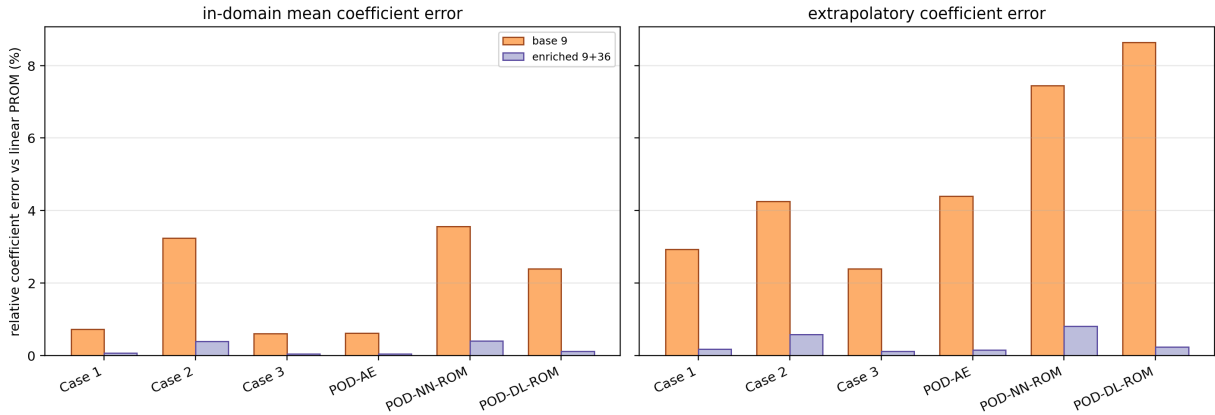


Figure 8: Effect of PROM data enrichment on online coefficient errors. The coefficient metric is stricter than the state metric because it measures the entire 151-coordinate trajectory against the linear PROM teacher.

The coefficient table confirms the state-error interpretation. The baseline POD-NN-ROM has an in-domain mean coefficient error of 3.557% and 7.444% at the extrapolatory point. After enrichment these decrease to 0.398% and 0.805%, respectively. Case 2 with $n = 10$ improves similarly because it uses the same master map for the injected tail.

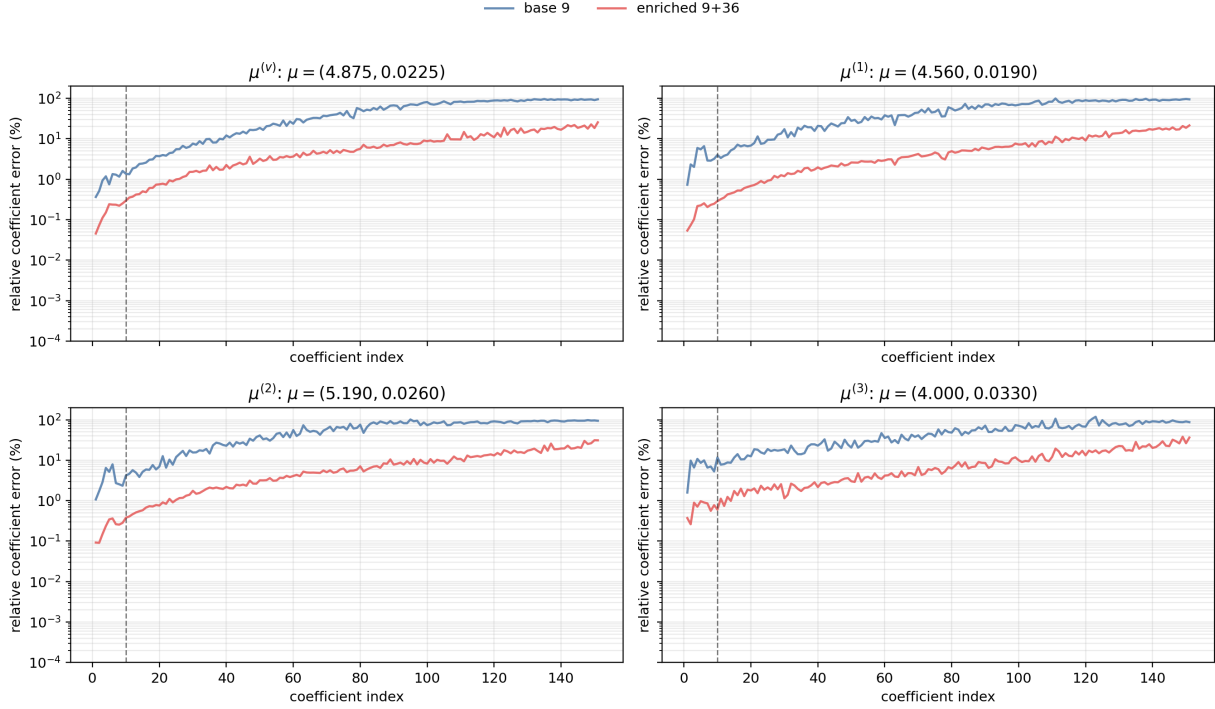


Figure 9: Per-coefficient relative error of the direct POD–NN–ROM coefficient prediction before and after PROM data enrichment. The vertical line marks coefficient 10. The plot uses the linear PROM coefficient trajectory as the reference.

Figure 9 explains why enrichment matters. With only nine trajectories, the high-index tail has relative errors close to order one even when the global state error is moderate. The enriched training set reduces those tail errors over most coefficients, while preserving the already accurate leading coefficients. This is precisely the behavior needed by Case 2: the PROM solve can recover the primary coordinates, but the residual still sees the injected secondary tail.

9 Conclusions

The PROM-only study supports the following conclusions.

1. The fixed MLSPG-sensitive linear PROM provides a reproducible coefficient teacher and state-accuracy reference for this benchmark.
2. With only nine training trajectories, direct parameter–time prediction of all 151 coefficients is the limiting task. This affects both the POD–NN–ROM and the Case 2 injected tail.
3. Solving more primary coordinates in Case 2 can help, but the residual solve remains coupled to the predicted secondary tail; therefore tail accuracy is still important.
4. PROM data enrichment changes the picture substantially. Keeping the architecture and basis fixed, the 9+36 PROM trajectory set reduces validation and online coefficient errors and brings the learned state errors close to the linear PROM level.

5. The next step should introduce residual sampling only after this PROM-level behavior is locked down, so that any additional error can be attributed to the sampled online residual rather than to the learned coefficient maps.